UNIVERSITY INSTITUTE of COMPUTING
Asia's Fastest Growing University

NAAC GRADE A+
Accredited University

Experiment- 7

Branch: MCA (AI&ML)	**Semester: 2**
Student Name: Ajay Rakwal	**UID:** 25MCI10329
Subject Name: Technical Training	**Subject Code**: 25CAP-652
Section/Group: MAM-1(A)	**Date of Performance:** 31-03-2026

Aim of the program:

Implementation of joins in PostgreSQL (inner join ,left join,right join, self join and cross join)

Software Requirements

- **Operating System:** Windows / Linux
- **Database Management System:** MySQL / Oracle / PostgreSQL
- **SQL Interface:** MySQL Workbench /Web Based / pgAdmin

Objective

Apply joins to a real-world database schema (e.g., Students, Courses, Enrollments, Departments)

Procedure of the Practical

I. Open PostgreSQL (pgAdmin) and connect to the required database.
II. Create necessary tables such as Students, Courses, Enrollments, and Departments, and insert sample records.
III. Execute an INNER JOIN query to display students along with their enrolled courses.

IV. Use a LEFT JOIN to find students who are not enrolled in any course.

V. Apply a RIGHT JOIN to display all courses, including those without enrolled students.

VI. Use SELF JOIN or multiple joins to display students along with their department information.

VII. Execute a CROSS JOIN to generate all possible combinations of students and courses and analyze the output.

Practical / Experiment Steps

```
--creating tables
CREATE TABLE departments (
    department_id SERIAL PRIMARY KEY,
    department_name VARCHAR(40)
);
CREATE TABLE students (
    student_id SERIAL PRIMARY KEY,
    name VARCHAR(40),
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(department_id)
);
CREATE TABLE courses (
    course_id SERIAL PRIMARY KEY,
    course_name VARCHAR(40)
);
```

```
CREATE TABLE enrollments (
    student_id INT,
    course_id INT,
    PRIMARY KEY (student_id, course_id),
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    FOREIGN KEY (course_id) REFERENCES courses(course_id)
);

--inserting values into tables
INSERT INTO departments (department_name) VALUES
('Computer Science'),
('Mechanical'),
('Electrical');
INSERT INTO students (name, department_id) VALUES
('Ajay', 1),
('Jatin', 2),
('Purnima', 1),
('Yuvraj', 3),
('Daniel', NULL);


INSERT INTO courses (course_name) VALUES
('DBMS'),
('OS'),
```

```
('Maths'),
('AI');
INSERT INTO enrollments (student_id, course_id) VALUES
(1, 1),
(1, 2),
(2, 3),
(3, 1);

--students with their enrolled courses (INNER JOIN)
SELECT
    s.student_id,
    s.name,
    c.course_name
FROM students s
INNER JOIN enrollments e
    ON s.student_id = e.student_id
INNER JOIN courses c
    ON e.course_id = c.course_id;

--students not enrolled in any course (LEFT JOIN).
SELECT
    s.student_id,
    s.name
FROM students s
```

```
LEFT JOIN enrollments e
    ON s.student_id = e.student_id
WHERE e.student_id IS NULL;


-- all courses with or without enrolled students (RIGHT JOIN).
SELECT
    c.course_name,
    s.name AS student_name
FROM enrollments e
RIGHT JOIN courses c
    ON e.course_id = c.course_id
LEFT JOIN students s
    ON e.student_id = s.student_id;


--students with department info using SELF JOIN or multiple joins.
SELECT
    s1.name AS student1,
    s2.name AS student2
FROM students s1
JOIN students s2
    ON s1.department_id = s2.department_id
WHERE s1.student_id <> s2.student_id;
```

```
--all possible student-course combinations (CROSS JOIN)
SELECT
    s.name,
    c.course_name
FROM students s
CROSS JOIN courses c;
```

I/O Analysis (Input / Output)

Input

- Base table creation queries for departments, students, courses, and enrollments
- Sample data insertion commands for all tables
- INNER JOIN query to display students with their enrolled courses
- LEFT JOIN query to find students not enrolled in any course
- RIGHT JOIN query to display all courses with or without enrolled students
- SELF JOIN query to find students belonging to the same department
- CROSS JOIN query to generate all possible student-course combinations

Output

- Base tables created successfully with primary and foreign key constraints
- Sample data inserted into all tables successfully
- INNER JOIN displaying students along with their enrolled courses
- LEFT JOIN identifying students with no course enrollments
- RIGHT JOIN showing all courses including those without students
- SELF JOIN displaying pairs of students from the same department
- CROSS JOIN generating all possible combinations of students and courses
- Effective demonstration of different JOIN operations for relational data retrieval

OUTPUT:

Table Created:

Data Output | Messages | Notifications

```
CREATE TABLE

Query returned successfully in 70 msec.
```

students with their enrolled courses (INNER JOIN):

Data Output | Messages | Notifications

	student_id integer	name character varying (40)	course_name character varying (40)
1	1	Ajay	DBMS
2	1	Ajay	OS
3	2	Jatin	Maths
4	3	Purnima	DBMS

-- students not enrolled in any course (LEFT JOIN).

Data Output | Messages | Notifications

	student_id [PK] integer	name character varying (40)
1	5	Daniel
2	4	Yuvraj

--students with department info using SELF JOIN or multiple joins.

Data Output | Messages | Notifications

	student1 character varying (40)	student2 character varying (40)
1	Ajay	Purnima
2	Purnima	Ajay

--all possible student-course combinations (CROSS JOIN)

Data Output | Messages | Notifications

	name character varying (40)	course_name character varying (40)
1	Ajay	DBMS
2	Jatin	DBMS
3	Purnima	DBMS
4	Yuvraj	DBMS
5	Daniel	DBMS
6	Ajay	OS
7	Jatin	OS
8	Purnima	OS
9	Yuvraj	OS
10	Daniel	OS
11	Ajay	Maths
12	Jatin	Maths
13	Purnima	Maths
14	Yuvraj	Maths
15	Daniel	Maths
16	Ajay	AI
17	Jatin	AI
18	Purnima	AI
19	Yuvraj	AI
20	Daniel	AI

Learning Outcome:

- Understand the concept and purpose of different types of joins in SQL.
- Learn how to apply INNER, LEFT, RIGHT, SELF, and CROSS JOINs in PostgreSQL.
- Gain practical experience in retrieving related data from multiple tables.
- Develop the ability to handle real-world relational database scenarios using joins.
- Understand how joins help in data integration and query optimization in database systems.